Parte 1: Questões Teóricas

1. A Orientação a Objetos é um paradigma de programação que organiza o código em estruturas que combinam dados (atributos) e comportamentos (métodos). Cite os quatro pilares da Orientação a Objetos e explique qual é o objetivo principal do **Encapsulamento**.

2. Uma das formas mais fáceis de entender a Orientação a Objetos é através de analogias do mundo real. Utilizando a analogia da "planta de uma casa", explique a diferença conceitual entre uma **Classe** e um **Objeto**.

3. O controle de acesso aos dados é uma parte essencial do design de software. Em Java, qual modificador de acesso devemos usar em um atributo para garantir que ele seja acessado apenas dentro da própria classe? Além disso, cite qual é a boa prática recomendada para permitir que outras partes do programa leiam e modifiquem esse valor com segurança.

4. O reaproveitamento de código em Java é frequentemente alcançado através da Herança. Ao criarmos uma classe filha, podemos construir construtores específicos para ela. Neste contexto, para que serve a chamada do método `super()` dentro do construtor de uma classe filha em Java?

5. O padrão de projeto criacional **Singleton** é utilizado para garantir que uma classe tenha apenas uma única instância e proveja um acesso global a ela. Na implementação clássica desse padrão em Java, o método construtor da classe deve ser declarado com qual modificador de acesso e por qual motivo?

Parte 2: Exercícios Práticos (Algoritmos)

6. Modelagem de Classe, Sobrecarga e Encapsulamento Elabore um algoritmo em Java que modele uma classe `Produto`. Siga os requisitos abaixo:

- Crie os atributos privados `nome` (String) e `preco` (double).
- Crie um construtor parametrizado que receba e inicialize o nome e o preço.
- Crie um construtor padrão (sem argumentos) que utilize a palavra-chave `this()` para encadear a chamada ao construtor parametrizado, passando os valores "Produto Sem Nome" e `0.0`.
- Crie métodos *getters* e *setters* para os dois atributos.
- Em um método `main`, instancie um objeto dessa classe usando o construtor padrão e outro usando o construtor parametrizado.

7. Herança e Sobrescrita (Override) Crie um algoritmo em Java para demonstrar o pilar da Herança e do Polimorfismo.

- Crie uma classe pai chamada `Animal` com um atributo `nome` (String) e um construtor para inicializá-lo.
- Na classe `Animal`, crie um método `emitirSom()` que imprima no console: "Som genérico de animal".
- Crie uma classe filha chamada `Gato` que herde da classe pai utilizando a palavra-chave `extends`.
- No construtor de `Gato`, utilize `super()` para repassar o nome para a classe pai.
- Na classe `Gato`, sobrescreva (utilizando a anotação `@Override`) o método `emitirSom()` para que ele imprima "Miau!".